

preprocessing_life_style_data_v2.12

November 26, 2025

1 Preprocessing - life_style_data.duckdb

1.1 1. Importation des librairies essentielles

```
[138]: import duckdb
import pandas as pd
import numpy as np
from pathlib import Path
import joblib, json
from datetime import datetime

import matplotlib.pyplot as plt
from sklearn.preprocessing import OrdinalEncoder
from sklearn.preprocessing import StandardScaler, FunctionTransformer
from sklearn.model_selection import train_test_split, GridSearchCV, KFold
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import BaggingRegressor
from sklearn.ensemble import GradientBoostingRegressor

from sklearn.pipeline import Pipeline

import sqlite3
```

1.2 2. Variables globales

```
[139]: # Définition des chemins (adaptés à ton arborescence GitHub)
# Notebook : C:\Users\fback\Desktop\Projets\Dev\GitHub\train.
# ↪ me\src\notebooks\preprocessing\life_style_data
# Base      : C:\Users\fback\Desktop\Projets\Dev\GitHub\train.
# ↪ me\src\data\raw\life_style_data

# Le notebook s'exécute depuis son répertoire → on peut repartir du cwd
current_dir = Path.cwd()
project_root = current_dir.parents[3] # remonte jusqu'à "train.me"
```

```
# Chemins complets
db_path = project_root / "src" / "data" / "raw" / "life_style_data" / "life_style_data"
export_dir = project_root / "src" / "data" / "processed" / "life_style_data"
model_dir = project_root / "src" / "models" / "v1" / "life_style_data"
```

1.3 3. Éviter de réutiliser d'anciens artefacts

```
[140]: for p in ["model.joblib", "feature_scaler.joblib", "target_scaler.joblib"]:
        f = (model_dir / p)
        if f.exists():
            f.unlink()
```

1.4 4. Nettoyage des données

1.4.1 4.1. Contexte & objectifs

Ce chapitre prépare le dataset `life_style_data` pour la modélisation. Objectifs : - Corriger les types (notamment les décimales avec virgules) ; - Quantifier et traiter les valeurs manquantes ; - Détecter/supprimer les doublons ; - Poser des règles de plausibilité métier (sans censurer) ; - Produire un rapport de nettoyage et des artefacts (dataset nettoyé).

Entrée : table DuckDB `life_style_data` (chargée via DBBeaver). Sorties : `df_clean` (Parquet/CSV), tableaux de suivi (manquants, conversions, doublons, flags).

1.4.2 4.2. Imports, paramètres, chargement depuis DuckDB

```
[141]: # Connexion à la base DuckDB
con = duckdb.connect(str(db_path))
# Connexion SQLite
con_sqlite = sqlite3.connect(export_dir / "life_style_data_val.db")

# Chargement de la table
df = con.execute("SELECT * FROM life_style_data").fetchdf()

# Extraire les 10 premières lignes
df_sample = df.head(10).copy()
df = df.iloc[10:].reset_index(drop=True)

# Écriture du sample dans la base SQLite
table_name = "sample"
df_sample.to_sql(table_name, con_sqlite, if_exists="replace", index=False)

print(f" Échantillon extrait : {df_sample.shape[0]} lignes")
print(f" Données restantes : {df.shape[0]} lignes")

# Vérification du chargement
shape_before = df.shape
```

```
print(f" Données chargées : {shape_before[0]} lignes, {shape_before[1]} \
↳ colonnes")
display(df.head(3))
```

Échantillon extrait : 10 lignes
Données restantes : 19990 lignes
Données chargées : 19990 lignes, 54 colonnes

	Age	Gender	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	Resting_BPM	\
0	50.51	Male	86.37	1.63	160.92	143.20	57.87	
1	47.77	Female	83.89	1.82	176.36	148.74	69.91	
2	19.00	Male	82.66	1.96	173.75	168.71	62.16	

	Session_Duration (hours)	Calories_Burned	Workout_Type	...	\
0	1.69	1974.26	Strength	...	
1	1.91	2733.40	HIIT	...	
2	1.89	2700.62	HIIT	...	

	cal_from_macros	pct_carbs	protein_per_kg	pct_HRR	pct_maxHR	\
0	2065.28	0.501588	1.189302	0.828045	0.889883	
1	1848.79	0.500133	1.111813	0.740535	0.843389	
2	1446.67	0.502727	0.866199	0.954835	0.970993	

	cal_balance	lean_mass_kg	expected_burn	Burns Calories (per 30 min)_bc	\
0	501.74	61.260921	1203.9560	9.983135e+19	
1	-501.40	64.050587	1379.5166	1.116947e+20	
2	-131.62	62.715145	1353.0888	1.039329e+20	

	Burns_Calories_Bin
0	High
1	Very High
2	High

[3 rows x 54 columns]

1.4.3 4.3. Audit qualité initial

On dresse un état des lieux : - Répartition des types et cardinalités ; - Valeurs manquantes (%) par colonne ; - Doublons potentiels.

Cet instantané nous sert de référence avant toute modification.

```
[142]: # Audit qualité initial
missing_pct = df.isna().mean().sort_values(ascending=False) * 100
dup_count = df.duplicated().sum()

summary_types = pd.DataFrame({
    "dtype": df.dtypes.astype(str),
    "n_unique": df.nunique(),
```

```

    "missing_%": df.isna().mean() * 100
}).sort_values("missing_%", ascending=False)

print(f"• Doublons détectés (avant): {dup_count}")
display(missing_pct.to_frame("missing_%").head(20))
display(summary_types.head(20))

```

• Doublons détectés (avant): 0

	missing_%
Age	0.0
Body Part	0.0
prep_time_min	0.0
cook_time_min	0.0
rating	0.0
Name of Exercise	0.0
Sets	0.0
Reps	0.0
Benefit	0.0
Burns Calories (per 30 min)	0.0
Target Muscle Group	0.0
Equipment Needed	0.0
Difficulty Level	0.0
Type of Muscle	0.0
Gender	0.0
Workout	0.0
BMI_calc	0.0
cal_from_macros	0.0
pct_carbs	0.0
protein_per_kg	0.0

	dtype	n_unique	missing_%
Age	float64	3950	0.0
Body Part	object	7	0.0
prep_time_min	float64	5384	0.0
cook_time_min	float64	9405	0.0
rating	float64	414	0.0
Name of Exercise	object	55	0.0
Sets	float64	30	0.0
Reps	float64	654	0.0
Benefit	object	49	0.0
Burns Calories (per 30 min)	float64	5765	0.0
Target Muscle Group	object	36	0.0
Equipment Needed	object	20	0.0
Difficulty Level	object	3	0.0
Type of Muscle	object	13	0.0
Gender	object	2	0.0
Workout	object	53	0.0
BMI_calc	float64	18150	0.0

cal_from_macros	float64	18798	0.0
pct_carbs	float64	19972	0.0
protein_per_kg	float64	19976	0.0

1.4.4 4.4. Correction de types (décimales à virgule, espaces insécables)

Dans de nombreux exports, les colonnes numériques arrivent en texte (object) avec : - des virgules à la place du point décimal ; - des espaces insécables (00A0) ou espaces.

On corrige proprement pour garantir la cohérence numérique.

```
[143]: # Conversion robuste des colonnes numériques candidates (si présentes)
num_candidates = [
    "Age", "Weight (kg)", "Height (m)", "Max_BPM", "Avg_BPM", "Resting_BPM",
    "Session_Duration (hours)", "Fat_Percentage", "Water_Intake (liters)",
    "Workout_Frequency (days/week)", "BMI", "Carbs", "Proteins", "Fats", "Calories",
    ↵
    ↵ "sugar_g", "sodium_mg", "cholesterol_mg", "serving_size_g", "prep_time_min", "cook_time_min",
    ↵
    ↵ "rating", "BMI_calc", "cal_from_macros", "pct_carbs", "protein_per_kg", "pct_HRR", "pct_maxHR",
    "cal_balance", "lean_mass_kg", "expected_burn", "Burns Calories (per 30↵
    ↵ min)", "Calories_Burned"
]

convert_log = []
df_conv = df.copy()

for c in [col for col in num_candidates if col in df_conv.columns]:
    if df_conv[c].dtype == "object":
        before_na = df_conv[c].isna().sum()
        df_conv[c] = (df_conv[c]
            .astype(str)
            .str.replace(",", ".", regex=False) # virgule point
            .str.replace("\u00A0", "", regex=False) # espace insécable
            .str.strip()
        )
        df_conv[c] = pd.to_numeric(df_conv[c], errors="coerce")
        after_na = df_conv[c].isna().sum()
        convert_log.append({"column": c, "was_object": True, "na_before": ↵
            ↵ before_na, "na_after": after_na})

convert_report = pd.DataFrame(convert_log).sort_values("column") if convert_log ↵
    ↵ else pd.DataFrame(columns=["column", "was_object", "na_before", "na_after"])
display(convert_report)
```

Empty DataFrame

Columns: [column, was_object, na_before, na_after]

Index: []

1.4.5 4.5. Règles de plausibilité (flags non bloquants)

On applique des bornes métier pour repérer des valeurs possiblement aberrantes. On ne supprime pas : on flague pour informer la modélisation et documenter les décisions.

Exemples (paramétrables) : - Age [10, 90] - Max_BPM [80, 230] - Session_Duration (hours) [0.1, 6] - BMI [10, 60]

```
[144]: # Flags de plausibilité (ne supprime rien)
df_flag = df_conv.copy()

def safe_between(s, low, high):
    return s.between(low, high) if s.notna().any() else pd.Series(False,
    ↪index=s.index)

flags = {}
if "Age" in df_flag: flags["Age_out"] =
    ↪safe_between(df_flag["Age"], 10, 90)
if "Max_BPM" in df_flag: flags["BPM_out"] =
    ↪safe_between(df_flag["Max_BPM"], 80, 230)
if "Session_Duration (hours)" in df_flag: flags["SessDur_out"] =
    ↪safe_between(df_flag["Session_Duration (hours)"], 0.1, 6)
if "BMI" in df_flag: flags["BMI_out"] =
    ↪safe_between(df_flag["BMI"], 10, 60)

flag_counts = {k: v.fillna(False).sum() for k, v in flags.items()}
flag_table = pd.DataFrame.from_dict(flag_counts, orient="index",
    ↪columns=["nb_outliers"]).sort_index()
display(flag_table)
```

	nb_outliers
Age_out	0
BMI_out	0
BPM_out	0
SessDur_out	0

1.4.6 4.6. Imputation des valeurs manquantes (baseline)

Stratégie simple et robuste (baseline reproductible) : - Numériques → médiane (moins sensible aux outliers) ; - Catégorielles → mode (valeur la plus fréquente, sinon “Unknown”).

Cette baseline est suffisante pour itérer ; on pourra spécialiser par la suite.

```
[145]: # Imputation des manquants
df_imp = df_flag.copy()

num_cols = df_imp.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = [c for c in df_imp.columns if c not in num_cols]
```

```

# Numériques → médiane
for c in num_cols:
    if df_imp[c].isna().any():
        df_imp[c] = df_imp[c].fillna(df_imp[c].median())

# Catégorielles → mode (ou "Unknown")
for c in cat_cols:
    if df_imp[c].isna().any():
        m = df_imp[c].mode(dropna=True)
        df_imp[c] = df_imp[c].fillna(m.iloc[0] if not m.empty else "Unknown")

missing_before = missing_pct # gardé depuis 1.2
missing_after = df_imp.isna().mean().sort_values(ascending=False) * 100

print(" Imputation effectuée.")
display(pd.DataFrame({"missing_before_%": missing_before}).head(10))
display(pd.DataFrame({"missing_after_%": missing_after}).head(10))

```

Imputation effectuée.

	missing_before_%
Age	0.0
Body Part	0.0
prep_time_min	0.0
cook_time_min	0.0
rating	0.0
Name of Exercise	0.0
Sets	0.0
Reps	0.0
Benefit	0.0
Burns Calories (per 30 min)	0.0

	missing_after_%
Age	0.0
Body Part	0.0
prep_time_min	0.0
cook_time_min	0.0
rating	0.0
Name of Exercise	0.0
Sets	0.0
Reps	0.0
Benefit	0.0
Burns Calories (per 30 min)	0.0

1.4.7 4.7. Doublons & incohérences simples

On élimine les doublons stricts (lignes identiques).

Cette opération stabilise la taille du corpus et évite des biais à l'entraînement.

```
[146]: # Doublons
rows_before = df_imp.shape[0]
dup_total = df_imp.duplicated().sum()
df_clean = df_imp.drop_duplicates().reset_index(drop=True)
rows_after = df_clean.shape[0]

clean_report = pd.DataFrame({
    "rows_before": [rows_before],
    "rows_after": [rows_after],
    "duplicates_removed": [dup_total],
    "cols": [df_clean.shape[1]]
})
display(clean_report)
```

	rows_before	rows_after	duplicates_removed	cols
0	19990	19990	0	54

1.4.8 4.8. Rapport de nettoyage

Nous consolidons les éléments clés pour traçabilité et communication : - Colonnes converties object → numeric ; - Avant/Après des valeurs manquantes ; - Nombre de doublons supprimés ; - Flags de plausibilité relevés.

Ce rapport sert de référence pour justifier les choix et préparer la suite (encodage, scaling).

```
[147]: # Rapport consolidé
sections = {
    "convert_report": convert_report,
    "missing_before_% (top 20)": missing_before.to_frame("missing_%").head(20),
    "missing_after_% (top 20)": missing_after.to_frame("missing_%").head(20),
    "plausibility_flags": flag_table,
    "clean_report": clean_report
}

for title, df_sec in sections.items():
    print("\n" + "="*30 + f" {title} " + "="*30)
    display(df_sec)
```

===== convert_report =====

```
Empty DataFrame
Columns: [column, was_object, na_before, na_after]
Index: []
```

===== missing_before_% (top 20)
=====

	missing_%
Age	0.0

Body Part	0.0
prep_time_min	0.0
cook_time_min	0.0
rating	0.0
Name of Exercise	0.0
Sets	0.0
Reps	0.0
Benefit	0.0
Burns Calories (per 30 min)	0.0
Target Muscle Group	0.0
Equipment Needed	0.0
Difficulty Level	0.0
Type of Muscle	0.0
Gender	0.0
Workout	0.0
BMI_calc	0.0
cal_from_macros	0.0
pct_carbs	0.0
protein_per_kg	0.0

===== missing_after_% (top 20)
=====

	missing_%
Age	0.0
Body Part	0.0
prep_time_min	0.0
cook_time_min	0.0
rating	0.0
Name of Exercise	0.0
Sets	0.0
Reps	0.0
Benefit	0.0
Burns Calories (per 30 min)	0.0
Target Muscle Group	0.0
Equipment Needed	0.0
Difficulty Level	0.0
Type of Muscle	0.0
Gender	0.0
Workout	0.0
BMI_calc	0.0
cal_from_macros	0.0
pct_carbs	0.0
protein_per_kg	0.0

===== plausibility_flags =====

nb_outliers

Age_out	0
BMI_out	0
BPM_out	0
SessDur_out	0

```
===== clean_report =====
      rows_before  rows_after  duplicates_removed  cols
0          19990          19990                   0    54
```

1.4.9 4.9. Export des artefacts (dataset nettoyé)

On persiste le dataset nettoyé pour les étapes suivantes (encodage, scaling, split) et pour reproductibilité : - life_style_data_clean.parquet (format recommandé) ; - Optionnel : life_style_data_clean.csv.

```
[148]: # Export artefacts
parquet_path = export_dir / "life_style_data_clean.parquet"

df_clean.to_parquet(parquet_path, index=False, engine="fastparquet")

print(f" Export parquet : {parquet_path}")
print(f" Shape final      : {df_clean.shape}")
```

```
Export parquet : c:\Users\fbback\Desktop\Projets\Dev\GitHub\train.me\src\data\p
rocessed\life_style_data\life_style_data_clean.parquet
Shape final      : (19990, 54)
```

1.4.10 4.9. Définition des champs métier pour l'entraînement

```
[149]: # Charger les artefacts nettoyés
df_features = pd.read_parquet(parquet_path)

print(f" Dataset chargé : {df_features.shape[0]} lignes, {df_features.
↪shape[1]} colonnes")
print(df_features.head(3))

# Filtrer uniquement les features "métier" retenues
# selected_features = [
#     # Prédiction target
#     "Calories_Burned",

#     # Core
#     "Age", "Gender", "Weight (kg)", "Max_BPM", "Avg_BPM", "Resting_BPM",
#     "Session_Duration (hours)", "Workout_Type", "Experience_Level",
#     "Difficulty_Level", "Body_Part", "Equipment_Needed",
#     "Workout_Frequency (days/week)", "Water_Intake (liters)",
#     "Fat_Percentage", "diet_type", "meal_type",
```

```

#     # Nutrition
#     "Calories", "Carbs", "Proteins", "Fats",

#     # Optionnels (si présents dans ton dataset)
#     "BMI", "sugar_g", "sodium_mg", "cholesterol_mg", "serving_size_g",
#     "Name of Exercise", "Sets", "Reps", "Target Muscle Group",
#     "Type of Muscle", "Benefit", "rating",
#     "cooking_method", "prep_time_min", "cook_time_min"
# ]
selected_features = [
    # Prédiction target
    "Calories_Burned",

    # Core
    "Age", "Gender", "Weight (kg)", "Height (m)", "Max_BPM", "Avg_BPM",
    ↪ "Resting_BPM",
    "Session_Duration (hours)", "Experience_Level", "Workout_Type",
    "Difficulty_Level", "Body Part", "Equipment_Needed",
    "Workout_Frequency (days/week)", ↪ "#Water_Intake (liters)",
    ↪ "#Fat_Percentage", "diet_type", "meal_type",

    # "Age", "Weight (kg)", "Gender", "Experience_Level", "Workout_Frequency
    ↪ (days/week)", "Session_Duration (hours)", "Height (m)", "Max_BPM",
    ↪ "Avg_BPM", "Resting_BPM", "Difficulty_Level", "Body Part", "Equipment_Needed"
]

# Vérifier quelles colonnes sont présentes dans le DataFrame
print(df_features.columns)
existing_features = [col for col in selected_features if col in df_features.
    ↪ columns]
missing_features = [col for col in selected_features if col not in df_features.
    ↪ columns]

# Filtrage du DataFrame sur les colonnes existantes
df_features = df_features[existing_features]

print(f"Colonnes conservées : {len(existing_features)}")
if missing_features:
    print(f"/!\ Colonnes absentes du dataset : {missing_features}")

parquet_path = export_dir / "life_style_data_feature.parquet"
df_features.to_parquet(parquet_path, index=False, engine="fastparquet")

print(f"Export parquet : {parquet_path}")

```

```
print(f"Shape final      : {df_clean.shape}")
```

Dataset chargé : 19990 lignes, 54 colonnes

	Age	Gender	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	Resting_BPM	\
0	50.51	Male	86.37	1.63	160.92	143.20	57.87	
1	47.77	Female	83.89	1.82	176.36	148.74	69.91	
2	19.00	Male	82.66	1.96	173.75	168.71	62.16	

	Session_Duration (hours)	Calories_Burned	Workout_Type	...	\
0	1.69	1974.26	Strength	...	
1	1.91	2733.40	HIIT	...	
2	1.89	2700.62	HIIT	...	

	cal_from_macros	pct_carbs	protein_per_kg	pct_HRR	pct_maxHR	\
0	2065.28	0.501588	1.189302	0.828045	0.889883	
1	1848.79	0.500133	1.111813	0.740535	0.843389	
2	1446.67	0.502727	0.866199	0.954835	0.970993	

	cal_balance	lean_mass_kg	expected_burn	Burns Calories (per 30 min)_bc	\
0	501.74	61.260921	1203.9560	9.983135e+19	
1	-501.40	64.050587	1379.5166	1.116947e+20	
2	-131.62	62.715145	1353.0888	1.039329e+20	

	Burns_Calories_Bin
0	High
1	Very High
2	High

[3 rows x 54 columns]

```
Index(['Age', 'Gender', 'Weight (kg)', 'Height (m)', 'Max_BPM', 'Avg_BPM',
      'Resting_BPM', 'Session_Duration (hours)', 'Calories_Burned',
      'Workout_Type', 'Fat_Percentage', 'Water_Intake (liters)',
      'Workout_Frequency (days/week)', 'Experience_Level', 'BMI',
      'Daily meals frequency', 'Physical exercise', 'Carbs', 'Proteins',
      'Fats', 'Calories', 'meal_name', 'meal_type', 'diet_type', 'sugar_g',
      'sodium_mg', 'cholesterol_mg', 'serving_size_g', 'cooking_method',
      'prep_time_min', 'cook_time_min', 'rating', 'Name of Exercise', 'Sets',
      'Reps', 'Benefit', 'Burns Calories (per 30 min)', 'Target Muscle Group',
      'Equipment Needed', 'Difficulty Level', 'Body Part', 'Type of Muscle',
      'Workout', 'BMI_calc', 'cal_from_macros', 'pct_carbs', 'protein_per_kg',
      'pct_HRR', 'pct_maxHR', 'cal_balance', 'lean_mass_kg', 'expected_burn',
      'Burns Calories (per 30 min)_bc', 'Burns_Calories_Bin'],
      dtype='object')
```

Colonnes conservées : 15

Export parquet : c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\data\processed\life_style_data\life_style_data_feature.parquet

Shape final : (19990, 54)

1.4.11 4.10. Critères d'acceptation & décisions

- Types numériques/catégoriels conformes au dictionnaire ;
- Manquants résiduels documentés et seuils tolérés (par défaut, 5%) ;
- Doublons supprimés (ou chiffrés et justifiés si conservés) ;
- Flags de plausibilité suivis (pas de censure sans décision métier explicite).

Décisions à garder au journal : - Colonnes converties, colonnes fortement manquantes et stratégie appliquée ; - Seuils de plausibilité retenus et impact (aucune ligne supprimée à ce stade) ; - Lieux d'export des artefacts.

1.5 5. Encodage des variables catégorielles

1.5.1 5.1. Objectifs

Cette étape consiste à transformer les variables catégorielles du jeu de données nettoyé en représentations numériques exploitables par les algorithmes de Machine Learning. L'encodage rend les colonnes qualitatives compatibles avec les modèles tout en préservant l'information sémantique. Deux approches principales sont appliquées selon la nature des catégories : - Encodage ordinal pour les variables hiérarchiques (ex. niveau d'expérience) ; - Encodage one-hot pour les variables nominales (ex. type d'exercice, partie du corps, équipement).

1.5.2 5.2. Chargement du dataset nettoyé

```
[150]: # Charger les artefacts nettoyés
df_clean = pd.read_parquet(parquet_path)

print(f" Dataset chargé : {df_clean.shape[0]} lignes, {df_clean.shape[1]} ↵
↵ colonnes")
df_clean.head(3)
```

Dataset chargé : 19990 lignes, 15 colonnes

```
[150]:   Calories_Burned   Age  Gender  Weight (kg)  Height (m)  Max_BPM  Avg_BPM  \
0         1974.26  50.51   Male         86.37         1.63   160.92   143.20
1         2733.40  47.77  Female         83.89         1.82   176.36   148.74
2         2700.62  19.00   Male         82.66         1.96   173.75   168.71

   Resting_BPM  Session_Duration (hours)  Experience_Level  Workout_Type  \
0         57.87                      1.69                2.98   Strength
1         69.91                      1.91                3.01      HIIT
2         62.16                      1.89                2.99      HIIT

   Difficulty Level  Body Part  Equipment Needed  \
0   Intermediate     Abs      Wall
1     Advanced  Shoulders  Dumbbells
2     Advanced     Abs  Bench or Sturdy Surface

   Workout_Frequency (days/week)
0                             4.01
```

1	5.02
2	5.02

1.5.3 5.3. Identification des variables catégorielles

On repère les variables non numériques à encoder. L'objectif est de distinguer : - les variables nominales (non ordonnées), - les variables ordinales (possédant une hiérarchie logique).

```
[151]: # Identification des colonnes catégorielles
cat_cols = df_clean.select_dtypes(exclude=["number"]).columns.tolist()
print(f" {len(cat_cols)} variables catégorielles détectées :")
cat_cols
```

5 variables catégorielles détectées :

```
[151]: ['Gender', 'Workout_Type', 'Difficulty Level', 'Body Part', 'Equipment Needed']
```

1.5.4 5.4. Encodage ordinal

Les variables catégorielles ordonnées (par ex. Experience_Level, Difficulty Level) sont converties en valeurs entières selon leur progression logique.

Cette méthode conserve la hiérarchie implicite entre les modalités.

```
[152]: # Encodage ordinal sur la variable à hiérarchie logique : Difficulty Level
difficulty_map = {
    "Beginner": 0,
    "Intermediate": 1,
    "Advanced": 2,
}

if "Difficulty Level" in df_clean.columns:
    df_clean["Difficulty Level"] = (
        df_clean["Difficulty Level"]
        .map(difficulty_map)
        .astype(float)
    )
    print("Encodage ordinal appliqué sur 'Difficulty Level' :", difficulty_map)

gender_encoder = OrdinalEncoder()
df_clean[["Gender"]] = gender_encoder.fit_transform(df_features[["Gender"]])
```

Encodage ordinal appliqué sur 'Difficulty Level' : {'Beginner': 0, 'Intermediate': 1, 'Advanced': 2}

1.5.5 5.5. Encodage One-Hot

Les variables nominales (sans hiérarchie) sont transformées par One-Hot Encoding, créant une colonne binaire par modalité.

Cela permet d'éviter toute interprétation de relation d'ordre entre catégories.

```
[153]: df_encoded = pd.get_dummies(
        df_clean,
        columns=[col for col in cat_cols if col not in ["Experience_Level",
        ↪ "Difficulty_Level"]],
        drop_first=True
    )

print(f"One-Hot Encoding appliqué : {df_encoded.shape[1]} colonnes au total")
df_encoded.head(10)
```

One-Hot Encoding appliqué : 40 colonnes au total

```
[153]:
```

	Calories_Burned	Age	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	\
0	1974.26	50.51	86.37	1.63	160.92	143.20	
1	2733.40	47.77	83.89	1.82	176.36	148.74	
2	2700.62	19.00	82.66	1.96	173.75	168.71	
3	2345.20	38.01	57.67	1.56	184.91	132.43	
4	743.85	27.39	71.79	1.97	180.96	151.11	
5	1293.84	18.86	95.12	1.85	189.48	142.51	
6	2189.58	40.36	56.88	1.71	168.02	142.67	
7	1305.60	37.14	76.48	1.77	180.29	127.80	
8	803.36	18.93	72.56	1.79	177.48	124.41	
9	1544.40	46.00	60.05	1.68	164.91	147.10	

	Resting_BPM	Session_Duration (hours)	Experience_Level	Difficulty_Level	\
0	57.87		1.69	2.98	1.0
1	69.91		1.91	3.01	2.0
2	62.16		1.89	2.99	2.0
3	54.04		1.64	3.00	2.0
4	72.68		0.75	1.02	2.0
5	49.90		1.20	1.98	2.0
6	60.89		1.87	3.01	2.0
7	56.76		1.36	2.00	0.0
8	60.95		0.81	1.02	0.0
9	62.34		1.17	2.00	1.0

	Equipment_Needed_Kettlebell	Equipment_Needed_Low Bar or TRX	\
0	...	False	False
1	...	False	False
2	...	False	False
3	...	False	False
4	...	False	False
5	...	False	False
6	...	False	False
7	...	False	False
8	...	False	True

9	...	False	False
---	-----	-------	-------

	Equipment Needed_None or Dumbbell	Equipment Needed_None or Dumbbells \
0	False	False
1	False	False
2	False	False
3	False	True
4	False	False
5	False	False
6	False	False
7	False	False
8	False	False
9	False	False

	Equipment Needed_Parallel Bars or Chair	Equipment Needed_Pull-up Bar \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
5	False	False
6	False	False
7	False	False
8	False	False
9	False	False

	Equipment Needed_Resistance Band \
0	False
1	False
2	False
3	False
4	False
5	False
6	False
7	False
8	False
9	False

	Equipment Needed_Resistance Band or Cable Machine \
0	False
1	False
2	False
3	False
4	False
5	True
6	False
7	False

8		False
9		False
	Equipment Needed_Step or Box	Equipment Needed_Wall
0	False	True
1	False	False
2	False	False
3	False	False
4	False	False
5	False	False
6	False	False
7	False	False
8	False	False
9	False	True

[10 rows x 40 columns]

1.5.6 5.6. Vérification post-encodage

Après encodage : - Toutes les variables sont numériques ; - Le nombre de colonnes augmente (selon le nombre de modalités) ; - Aucun NaN ne doit être réintroduit.

```
[154]: print(f"Forme finale du dataset encodé : {df_encoded.shape}")
print(f"Valeurs manquantes totales : {df_encoded.isna().sum().sum()}")

# Aperçu d'un sous-ensemble des nouvelles colonnes encodées
df_encoded.filter(like="Workout_Type").head(10)
```

Forme finale du dataset encodé : (19990, 40)

Valeurs manquantes totales : 0

```
[154]: Workout_Type_HIIT Workout_Type_Strength Workout_Type_Yoga
0          False          True          False
1           True          False          False
2           True          False          False
3           True          False          False
4          False          True          False
5          False          True          False
6          False          True          False
7          False          False          False
8          False          True          False
9           True          False          False
```

1.5.7 5.7. Export du dataset encodé

On exporte le jeu de données final encodé pour les étapes suivantes : - Mise à l'échelle / normalisation - Split train/test

```
[155]: encoded_path = export_dir / "life_style_data_encoded.parquet"
df_encoded.to_parquet(encoded_path, index=False)
print(f"Export du dataset encodé : {encoded_path}")
```

Export du dataset encodé : c:\Users\fbback\Desktop\Projets\Dev\GitHub\train.me\src\data\processed\life_style_data\life_style_data_encoded.parquet

L'encodage garantit la compatibilité du jeu de données avec les algorithmes de Machine Learning.

Les variables catégorielles hiérarchiques ont été traitées par encodage ordinal, et les variables nominales par encodage one-hot, réduisant les risques de biais d'interprétation.

1.6 6. Mise à l'échelle / Normalisation des variables numériques

1.6.1 6.1. Objectifs

Cette étape vise à homogénéiser les échelles des variables numériques afin d'éviter qu'une variable à grande amplitude (ex. Calories ou Weight (kg)) ne domine les autres lors de l'apprentissage du modèle. Deux techniques sont généralement utilisées : - StandardScaler : recentre les données autour de 0 et réduit leur variance à 1 ; - MinMaxScaler : ramène les valeurs dans un intervalle [0,1].

Le choix du scaler dépend de la nature des modèles : - Les algorithmes linéaires ou ceux utilisant une distance (SVM, KNN, PCA) sont sensibles à l'échelle → StandardScaler privilégié ; - Les modèles arborescents (Random Forest, XGBoost) y sont peu sensibles.

```
[156]: encoded_path = export_dir / "life_style_data_encoded.parquet"

df_encoded = pd.read_parquet(encoded_path)
print(f"Dataset encodé chargé : {df_encoded.shape[0]} lignes, {df_encoded.
    ↳shape[1]} colonnes")
df_encoded.head(3)
```

Dataset encodé chargé : 19990 lignes, 40 colonnes

```
[156]:
```

	Calories_Burned	Age	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	\
0	1974.26	50.51	86.37	1.63	160.92	143.20	
1	2733.40	47.77	83.89	1.82	176.36	148.74	
2	2700.62	19.00	82.66	1.96	173.75	168.71	

	Resting_BPM	Session_Duration (hours)	Experience_Level	Difficulty Level	\
0	57.87		1.69	2.98	1.0
1	69.91		1.91	3.01	2.0
2	62.16		1.89	2.99	2.0

	Equipment Needed_Kettlebell	Equipment Needed_Low Bar or TRX	\
0	False	False	
1	False	False	
2	False	False	

	Equipment Needed_None or Dumbbell	Equipment Needed_None or Dumbbells \
0	False	False
1	False	False
2	False	False

	Equipment Needed_Parallel Bars or Chair	Equipment Needed_Pull-up Bar \
0	False	False
1	False	False
2	False	False

	Equipment Needed_Resistance Band \
0	False
1	False
2	False

	Equipment Needed_Resistance Band or Cable Machine \
0	False
1	False
2	False

	Equipment Needed_Step or Box	Equipment Needed_Wall
0	False	True
1	False	False
2	False	False

[3 rows x 40 columns]

1.6.2 6.2. Sélection des variables numériques

On identifie toutes les variables numériques du dataset afin d'appliquer la mise à l'échelle uniquement sur celles qui ne sont pas déjà binaires (0/1).

Les colonnes issues du One-Hot Encoding ne doivent pas être modifiées.

```
[157]: # Identifier toutes les colonnes numériques
num_cols = df_encoded.select_dtypes(include=[np.number]).columns.tolist()

# Exclure les colonnes binaires (issues du One-Hot Encoding)
non_binary_num_cols = [col for col in num_cols if df_encoded[col].nunique() > 2]

print(f"Variables numériques à mettre à l'échelle : {len(non_binary_num_cols)}")
non_binary_num_cols[:15] # aperçu
```

Variables numériques à mettre à l'échelle : 11

```
[157]: ['Calories_Burned',
        'Age',
        'Weight (kg)',
        'Height (m)',
```

```
'Max_BPM',
'Avg_BPM',
'Resting_BPM',
'Session_Duration (hours)',
'Experience_Level',
'Difficulty Level',
'Workout_Frequency (days/week)']
```

1.6.3 6.3. Application du StandardScaler

Le StandardScaler est appliqué sur toutes les variables continues identifiées.

Cette normalisation rend la moyenne nulle et la variance unitaire, garantissant un traitement équilibré par les algorithmes sensibles à l'échelle des features.

```
[158]: scaler = StandardScaler()
scaled_values = scaler.fit_transform(df_encoded[non_binary_num_cols])

# Remplacer les colonnes originales par leurs versions normalisées
df_scaled = df_encoded.copy()
df_scaled[non_binary_num_cols] = scaled_values

print("Standardisation terminée.")
df_scaled[non_binary_num_cols].describe().T.head(10)
```

Standardisation terminée.

```
[158]:
```

	count	mean	std	min	25%	\
Calories_Burned	19990.0	-1.094783e-16	1.000025	-1.905592	-0.735383	
Age	19990.0	1.206750e-16	1.000025	-1.721249	-0.881770	
Weight (kg)	19990.0	1.597744e-16	1.000025	-1.639784	-0.743384	
Height (m)	19990.0	1.135304e-15	1.000025	-1.834886	-0.811578	
Max_BPM	19990.0	4.130318e-16	1.000025	-1.787597	-0.854658	
Avg_BPM	19990.0	-2.847147e-16	1.000025	-1.726579	-0.875041	
Resting_BPM	19990.0	9.568690e-16	1.000025	-1.743387	-0.855687	
Session_Duration (hours)	19990.0	2.317528e-16	1.000025	-2.254353	-0.613733	
Experience_Level	19990.0	3.284350e-16	1.000025	-1.099061	-1.085478	
Difficulty Level	19990.0	-2.985772e-17	1.000025	-1.226555	-1.226555	

	50%	75%	max
Calories_Burned	-0.096486	0.543406	3.207234
Age	0.084003	0.889639	1.718387
Weight (kg)	-0.184197	0.576066	2.685888
Height (m)	-0.103134	0.605310	2.258346
Max_BPM	0.021819	0.829238	1.715705
Avg_BPM	-0.050136	0.865880	1.831657
Resting_BPM	0.000457	0.808580	1.688048
Session_Duration (hours)	0.030796	0.587435	2.228054
Experience_Level	0.245683	0.286433	1.685511

Difficulty Level -0.000491 1.225574 1.225574

1.6.4 6.4. Vérification post-normalisation

Après mise à l'échelle : - La moyenne des colonnes est proche de 0 ; - L'écart-type est proche de 1 ; - Les colonnes binaires n'ont pas été modifiées.

```
[159]: # Vérification moyenne ~0 et std ~1
check_stats = df_scaled[non_binary_num_cols].agg(["mean", "std"]).T
check_stats.head(10)
```

```
[159]:
```

	mean	std
Calories_Burned	-1.094783e-16	1.000025
Age	1.206750e-16	1.000025
Weight (kg)	1.597744e-16	1.000025
Height (m)	1.135304e-15	1.000025
Max_BPM	4.130318e-16	1.000025
Avg_BPM	-2.847147e-16	1.000025
Resting_BPM	9.568690e-16	1.000025
Session_Duration (hours)	2.317528e-16	1.000025
Experience_Level	3.284350e-16	1.000025
Difficulty Level	-2.985772e-17	1.000025

1.6.5 6.5. Export du dataset normalisé

Le dataset final normalisé est sauvegardé pour la suite du pipeline : - life_style_data_scaled.parquet : pour la modélisation supervisée ; - life_style_data_scaled.csv : pour inspection rapide sous DBeaver.

```
[160]: scaled_path = export_dir / "life_style_data_scaled.parquet"
df_scaled.to_parquet(scaled_path, index=False)

print(f"Export parquet : {scaled_path}")
print(f"Shape final : {df_scaled.shape}")
```

```
Export parquet : c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\data\processed\life_style_data\life_style_data_scaled.parquet
Shape final : (19990, 40)
```

Critères de validation : - Moyenne 0 et écart-type 1 pour les colonnes normalisées ; - Aucune valeur manquante ou anormale après transformation ; - Colonnes binaires conservées inchangées ; - Fichiers d'export générés sans perte de structure.

Cette étape garantit une échelle cohérente entre toutes les variables continues, assurant un apprentissage plus stable et une meilleure convergence lors de la modélisation.

1.7 7. Séparation features/target et split en jeux d'entraînement et de test

1.7.1 7.1. Objectif

L'objectif de cette étape est de préparer les données d'apprentissage en distinguant : - les features (X) : variables explicatives, - la target (y) : variable cible à prédire.

Le jeu de données est ensuite découpé en deux sous-ensembles : - Jeu d'entraînement (train set) : pour ajuster les paramètres du modèle (généralement 80 %) ; - Jeu de test (test set) : pour évaluer sa capacité de généralisation (généralement 20 %).

Cette séparation est une pratique fondamentale du Machine Learning supervisé, garantissant la fiabilité de l'évaluation des performances.

```
[161]: scaled_path = export_dir / "life_style_data_scaled.parquet"

df_scaled = pd.read_parquet(scaled_path)

print(f"Dataset normalisé chargé : {df_scaled.shape[0]} lignes, {df_scaled.
↪shape[1]} colonnes")
df_scaled.head(3)
```

Dataset normalisé chargé : 19990 lignes, 40 colonnes

```
[161]:
```

	Calories_Burned	Age	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	\
0	1.382179	0.962278	0.588936	-0.732862	-1.647743	-0.035418	
1	2.893779	0.736106	0.471809	0.762742	-0.306534	0.352855	
2	2.828508	-1.638705	0.413717	1.864766	-0.533254	1.752461	

	Resting_BPM	Session_Duration (hours)	Experience_Level	Difficulty Level	\
0	-0.593630	1.261261	1.590428	-0.000491	
1	1.058288	1.905790	1.631178	1.225574	
2	-0.005031	1.847196	1.604011	1.225574	

...	Equipment Needed_Kettlebell	Equipment Needed_Low Bar or TRX	\
0	False	False	
1	False	False	
2	False	False	

	Equipment Needed_None or Dumbbell	Equipment Needed_None or Dumbbells	\
0	False	False	
1	False	False	
2	False	False	

	Equipment Needed_Parallel Bars or Chair	Equipment Needed_Pull-up Bar	\
0	False	False	
1	False	False	
2	False	False	

	Equipment Needed_Resistance Band	\
--	----------------------------------	---

```

0          False
1          False
2          False

Equipment Needed_Resistance Band or Cable Machine \
0          False
1          False
2          False

Equipment Needed_Step or Box   Equipment Needed_Wall
0          False              True
1          False              False
2          False              False

[3 rows x 40 columns]

```

1.7.2 7.2. Définition de la variable cible

La variable à prédire dans ce projet est Calories_Burned (y_calories_burned dans la table).

Elle représente la dépense calorique totale estimée selon les caractéristiques physiques et comportementales de l'utilisateur.

Toutes les autres colonnes servent de features (X).

```

[162]: # Identifier la colonne cible
target_col = "Calories_Burned"

# Vérification que la variable existe bien
assert target_col in df_scaled.columns, f"La colonne cible {target_col} est_
↳ introuvable."

# Séparation X (features) / y (target)
X = df_scaled.drop(columns=[target_col])
print(X)
y = df_scaled[target_col]
print(y)

print(f"Features (X) : {X.shape[1]} colonnes")
print(f"Target (y)   : {y.name}")

```

```

      Age  Weight (kg)  Height (m)  Max_BPM  Avg_BPM  Resting_BPM \
0    0.962278    0.588936   -0.732862 -1.647743 -0.035418   -0.593630
1    0.736106    0.471809    0.762742 -0.306534  0.352855    1.058288
2   -1.638705    0.413717    1.864766 -0.533254  1.752461   -0.005031
3   -0.069530   -0.766527   -1.283874  0.436170 -0.790238   -1.119116
4   -0.946155   -0.099658    1.943482  0.093050  0.518958    1.438340
...      ...      ...      ...      ...      ...      ...
19985  0.653561    1.152846    1.392470  1.677484  0.313607    0.209005

```

19986	0.126100	0.671586	1.156322	1.415149	-0.667589	-1.119116
19987	0.945769	-1.308239	-0.417998	-1.437528	0.996239	-0.075004
19988	1.114986	-1.397974	-0.811578	-0.053754	-1.575195	-0.180650
19989	-0.767033	-0.721187	-0.890294	1.579325	-1.539451	1.351902

	Session_Duration (hours)	Experience_Level	Difficulty_Level \
0	1.261261	1.590428	-0.000491
1	1.905790	1.631178	1.225574
2	1.847196	1.604011	1.225574
3	1.114777	1.617595	1.225574
4	-1.492636	-1.071895	1.225574
...	...	...	...
19985	-1.434043	-1.099061	1.225574
19986	2.081571	1.604011	-0.000491
19987	0.294467	0.259267	-1.226555
19988	0.440951	-1.099061	-0.000491
19989	-0.174281	0.218517	1.225574

	Workout_Frequency (days/week) ...	Equipment_Needed_Kettlebell \
0	0.758953 ...	False
1	1.867380 ...	False
2	1.867380 ...	False
3	1.845431 ...	False
4	-0.338500 ...	False
...		...
19985	-0.327525 ...	False
19986	1.856406 ...	False
19987	0.769928 ...	False
19988	-0.327525 ...	False
19989	-0.338500 ...	False

	Equipment_Needed_Low Bar or TRX	Equipment_Needed_None or Dumbbell \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
19985	False	False
19986	False	False
19987	False	False
19988	False	False
19989	False	False

	Equipment_Needed_None or Dumbbells \
0	False
1	False
2	False

3	True
4	False
...	...
19985	False
19986	False
19987	False
19988	False
19989	False

	Equipment Needed_Parallel Bars or Chair	Equipment Needed_Pull-up Bar \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
19985	False	False
19986	False	False
19987	False	False
19988	False	True
19989	False	True

	Equipment Needed_Resistance Band \
0	False
1	False
2	False
3	False
4	False
...	...
19985	False
19986	False
19987	False
19988	False
19989	False

	Equipment Needed_Resistance Band or Cable Machine \
0	False
1	False
2	False
3	False
4	False
...	...
19985	False
19986	False
19987	False
19988	False
19989	False

	Equipment Needed_Step or Box	Equipment Needed_Wall
0	False	True
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
19985	False	False
19986	False	False
19987	False	False
19988	False	False
19989	False	False

[19990 rows x 39 columns]

0	1.382179
1	2.893779
2	2.828508
3	2.120795
4	-1.067814

...	
19985	-1.032450
19986	2.037025
19987	0.375707
19988	-0.697650
19989	-0.832872

Name: Calories_Burned, Length: 19990, dtype: float64

Features (X) : 39 colonnes

Target (y) : Calories_Burned

1.7.3 7.3. Split train/test/val

Le split est effectué avec un ratio suivant : - Train (70 %) → apprentissage des poids du modèle - Validation (15 %) → sélection des hyperparamètres et early stopping - Test (15 %) → évaluation finale, non touché avant la fin du projet

```
[163]: # --- Charger les features BRUTES après encodage, avant scaling global ---
raw_feat_path = export_dir / "life_style_data_encoded.parquet" # dataset_
↳ encodé, non normalisé
df_feat_raw = pd.read_parquet(raw_feat_path)
print(df_feat_raw.columns)

feature_cols_raw = [
    "Age",
    "Weight (kg)",
    "Height (m)",
    "Gender_1.0",
    "Experience_Level",
    "Difficulty Level",
```

```

"Workout_Frequency (days/week)",
"Session_Duration (hours)",
"Max_BPM",
"Avg_BPM",
"Resting_BPM",
"Workout_Type_HIIT",
"Workout_Type_Strength",
"Workout_Type_Yoga",
"Body Part_Arms",
"Body Part_Back",
"Body Part_Chest",
"Body Part_Forearms",
"Body Part_Legs",
"Body Part_Shoulders",
"Equipment_Needed_Bench or Step",
"Equipment_Needed_Bench or Sturdy Surface",
"Equipment_Needed_Bench, Barbell",
"Equipment_Needed_Box or Platform",
"Equipment_Needed_Cable Machine",
"Equipment_Needed_Cable Machine or Resistance Band",
"Equipment_Needed_Dumbbells",
"Equipment_Needed_Dumbbells or Barbell",
"Equipment_Needed_Kettlebell",
"Equipment_Needed_Low Bar or TRX",
"Equipment_Needed_None or Dumbbell",
"Equipment_Needed_None or Dumbbells",
"Equipment_Needed_Parallel Bars or Chair",
"Equipment_Needed_Pull-up Bar",
"Equipment_Needed_Resistance Band",
"Equipment_Needed_Resistance Band or Cable Machine",
"Equipment_Needed_Step or Box",
"Equipment_Needed_Wall"
]

assert "Calories_Burned" in df_feat_raw.columns
assert set(feature_cols_raw) <= set(df_feat_raw.columns), \
    f"Colonnes manquantes dans df_feat_raw : {set(feature_cols_raw) - \
    ↪ set(df_feat_raw.columns)}"

# X/y bruts (unités réelles pour Age / Weight / Calories_Burned, binaire pour ↪
    ↪ Gender_1.0)
X_raw = df_feat_raw[feature_cols_raw].copy()
y_raw = df_feat_raw["Calories_Burned"].copy()

print(f"X_raw shape: {X_raw.shape} | y_raw shape: {y_raw.shape}")

# --- Split BRUT (puisque'on va fitter les scalers sur TRAIN uniquement) ---

```

```

X_temp_raw, X_test_raw, y_temp_raw, y_test_raw = train_test_split(
    X_raw, y_raw, test_size=0.15, random_state=42
)
X_train_raw, X_val_raw, y_train_raw, y_val_raw = train_test_split(
    X_temp_raw, y_temp_raw, test_size=0.1765, random_state=42
)

print(f"Train: {X_train_raw.shape} | Val: {X_val_raw.shape} | Test: {X_test_raw.
↪shape}")
print(f"y_train_raw min/max: {y_train_raw.min():.2f} / {y_train_raw.max():.2f}")

```

```

Index(['Calories_Burned', 'Age', 'Weight (kg)', 'Height (m)', 'Max_BPM',
      'Avg_BPM', 'Resting_BPM', 'Session_Duration (hours)',
      'Experience_Level', 'Difficulty_Level', 'Workout_Frequency (days/week)',
      'Gender_1.0', 'Workout_Type_HIIT', 'Workout_Type_Strength',
      'Workout_Type_Yoga', 'Body_Part_Arms', 'Body_Part_Back',
      'Body_Part_Chest', 'Body_Part_Forearms', 'Body_Part_Legs',
      'Body_Part_Shoulders', 'Equipment_Needed_Bench or Chair',
      'Equipment_Needed_Bench or Step',
      'Equipment_Needed_Bench or Sturdy Surface',
      'Equipment_Needed_Bench, Barbell', 'Equipment_Needed_Box or Platform',
      'Equipment_Needed_Cable Machine',
      'Equipment_Needed_Cable Machine or Resistance Band',
      'Equipment_Needed_Dumbbells', 'Equipment_Needed_Dumbbells or Barbell',
      'Equipment_Needed_Kettlebell', 'Equipment_Needed_Low Bar or TRX',
      'Equipment_Needed_None or Dumbbell',
      'Equipment_Needed_None or Dumbbells',
      'Equipment_Needed_Parallel Bars or Chair',
      'Equipment_Needed_Pull-up Bar', 'Equipment_Needed_Resistance Band',
      'Equipment_Needed_Resistance Band or Cable Machine',
      'Equipment_Needed_Step or Box', 'Equipment_Needed_Wall'],
      dtype='object')
X_raw shape: (19990, 38) | y_raw shape: (19990,)
Train: (13992, 38) | Val: (2999, 38) | Test: (2999, 38)
y_train_raw min/max: 323.11 / 2876.51

```

1.8 8. Fit des scalers sur TRAIN brut puis transform des 3 splits

```

[164]: feature_scaler = StandardScaler().fit(X_train_raw[feature_cols_raw])

X_train = pd.DataFrame(
    feature_scaler.transform(X_train_raw[feature_cols_raw]),
    columns=feature_cols_raw
)
X_val = pd.DataFrame(
    feature_scaler.transform(X_val_raw[feature_cols_raw]),
    columns=feature_cols_raw
)

```

```

)
X_test = pd.DataFrame(
    feature_scaler.transform(X_test_raw[feature_cols_raw]),
    columns=feature_cols_raw
)

target_scaler = StandardScaler().fit(y_train_raw.to_numpy().reshape(-1,1))
y_train_std = target_scaler.transform(y_train_raw.to_numpy().reshape(-1,1)).
    ↪ravel()
y_val_std   = target_scaler.transform(y_val_raw.to_numpy().reshape(-1,1)).
    ↪ravel()
y_test_std  = target_scaler.transform(y_test_raw.to_numpy().reshape(-1,1)).
    ↪ravel()

# Sauvegarde (une seule fois ici)
joblib.dump(feature_scaler, model_dir / "feature_scaler.joblib")
joblib.dump(target_scaler, model_dir / "target_scaler.joblib")
joblib.dump(gender_encoder, model_dir / "gender_encoder.joblib")

```

[164]: ['c:\\Users\\fback\\Desktop\\Projets\\Dev\\GitHub\\train.me\\src\\models\\v1\\life_style_data\\gender_encoder.joblib']

1.8.1 8.1. Vérification statistique du split

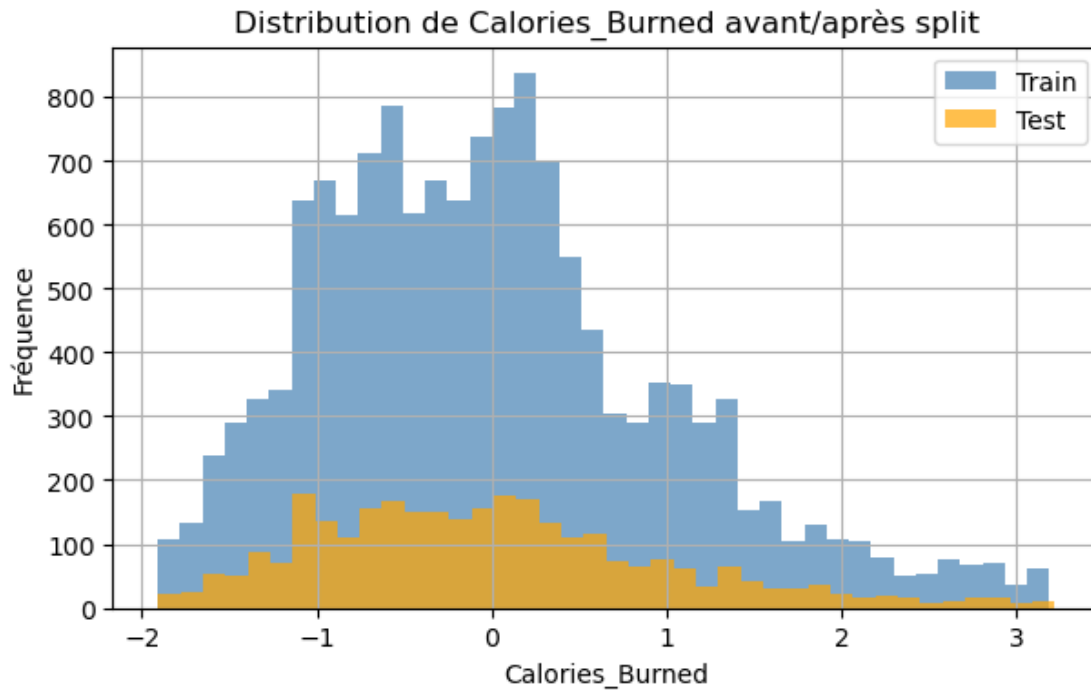
On vérifie que : - les proportions sont respectées (80/20), - la distribution statistique de la variable cible reste similaire entre les deux sous-ensembles (absence de biais de séparation).

```

[165]: plt.figure(figsize=(7,4))
plt.hist(y_train_std, bins=40, alpha=0.7, label="Train", color="steelblue")
plt.hist(y_test_std, bins=40, alpha=0.7, label="Test", color="orange")
plt.legend()
plt.title("Distribution de Calories_Burned avant/après split")
plt.xlabel("Calories_Burned")
plt.ylabel("Fréquence")
plt.grid(True)
plt.show()

print(f"Train mean: {y_train_std.mean():.2f}, Test mean: {y_test_std.mean():.
    ↪2f}")

```



Train mean: 0.00, Test mean: -0.01

1.8.2 8.2. Export des sous-ensembles

Les quatre sous-ensembles sont sauvegardés sous format Parquet, prêts à être utilisés pour l'entraînement et la validation des modèles.

```
[166]: # --- Export des X (déjà DataFrame)
X_train.to_parquet(export_dir / "X_train.parquet", index=False)
X_test.to_parquet(export_dir / "X_test.parquet", index=False)
X_val.to_parquet(export_dir / "X_val.parquet", index=False)

# --- Export des y standardisés (convertir ndarray → Series/DataFrame)
target_col_std = "Calories_Burned_std"

pd.Series(y_train_std, name=target_col_std).to_frame() \
    .to_parquet(export_dir / "y_train.parquet", index=False)

pd.Series(y_test_std, name=target_col_std).to_frame() \
    .to_parquet(export_dir / "y_test.parquet", index=False)

pd.Series(y_val_std, name=target_col_std).to_frame() \
    .to_parquet(export_dir / "y_val.parquet", index=False)
```

Synthèse & critères d'acceptation : - La variable cible Calories_Burned est correctement isolée.
 - Les proportions de split sont respectées (80 % / 20 %). - Les distributions train/test sont

similaires → absence de biais de sélection. - Les fichiers Parquet exportés garantissent la traçabilité et la reproductibilité du pipeline.

Cette étape finalise la préparation des données d'apprentissage, assurant une séparation claire entre entraînement et évaluation.

Elle constitue la base du bloc 5 — Feature Engineering et de l'entraînement des modèles supervisés.

1.9 9. Sélection ou création de features pertinentes

1.9.1 9.1. Objectif

Cette étape vise à optimiser la qualité des variables utilisées pour la modélisation, en éliminant les features redondantes ou peu informatives et en créant de nouvelles variables plus représentatives du phénomène étudié.

Deux approches sont utilisées : - Sélection de features : identification des variables les plus corrélées à la cible (feature importance). - Création de nouvelles features : dérivées ou combinaisons logiques des variables existantes (ratios, interactions, indicateurs composites).

```
[167]: X_train = pd.read_parquet(export_dir / "X_train.parquet")
X_test  = pd.read_parquet(export_dir / "X_test.parquet")
y_train = pd.read_parquet(export_dir / "y_train.parquet").squeeze() # ↪
      ↪conversion en Series
y_test  = pd.read_parquet(export_dir / "y_test.parquet").squeeze() # ↪conversion
      ↪en Series

print(f"Données chargées : {X_train.shape[0]} lignes, {X_train.shape[1]} ↪
      ↪colonnes")
print(f"Données chargées : {X_test.shape[0]} lignes, {X_test.shape[1]} ↪
      ↪colonnes")
```

Données chargées : 13992 lignes, 38 colonnes

Données chargées : 2999 lignes, 38 colonnes

1.9.2 9.2. Analyse de corrélation et importance des variables

On commence par une analyse de corrélation entre les variables numériques et la cible.

Cette étape permet de repérer les variables les plus influentes sur Calories_Burned.

Une méthode complémentaire basée sur un modèle de RandomForestRegressor est utilisée pour estimer l'importance relative des features.

```
[168]: # Calcul des corrélations (variables continues uniquement)
corr = X_train.corrwith(y_train).dropna().sort_values(ascending=False)
top_corr = corr.head(20)

plt.figure(figsize=(8,4))
top_corr.plot(kind="bar", color="steelblue")
```

```

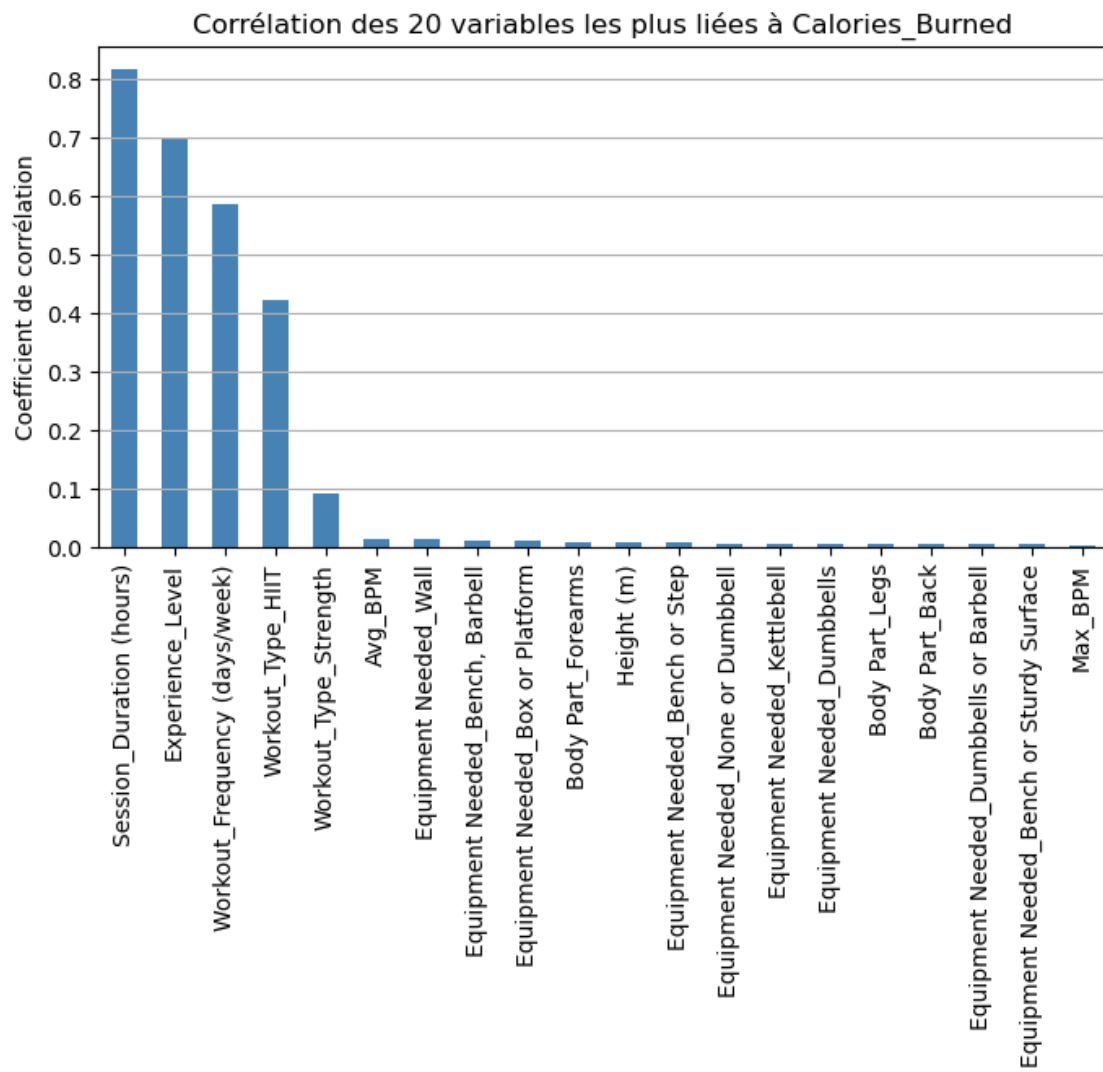
plt.title("Corrélation des 20 variables les plus liées à Calories_Burned")
plt.ylabel("Coefficient de corrélation")
plt.grid(True, axis='y')
plt.show()

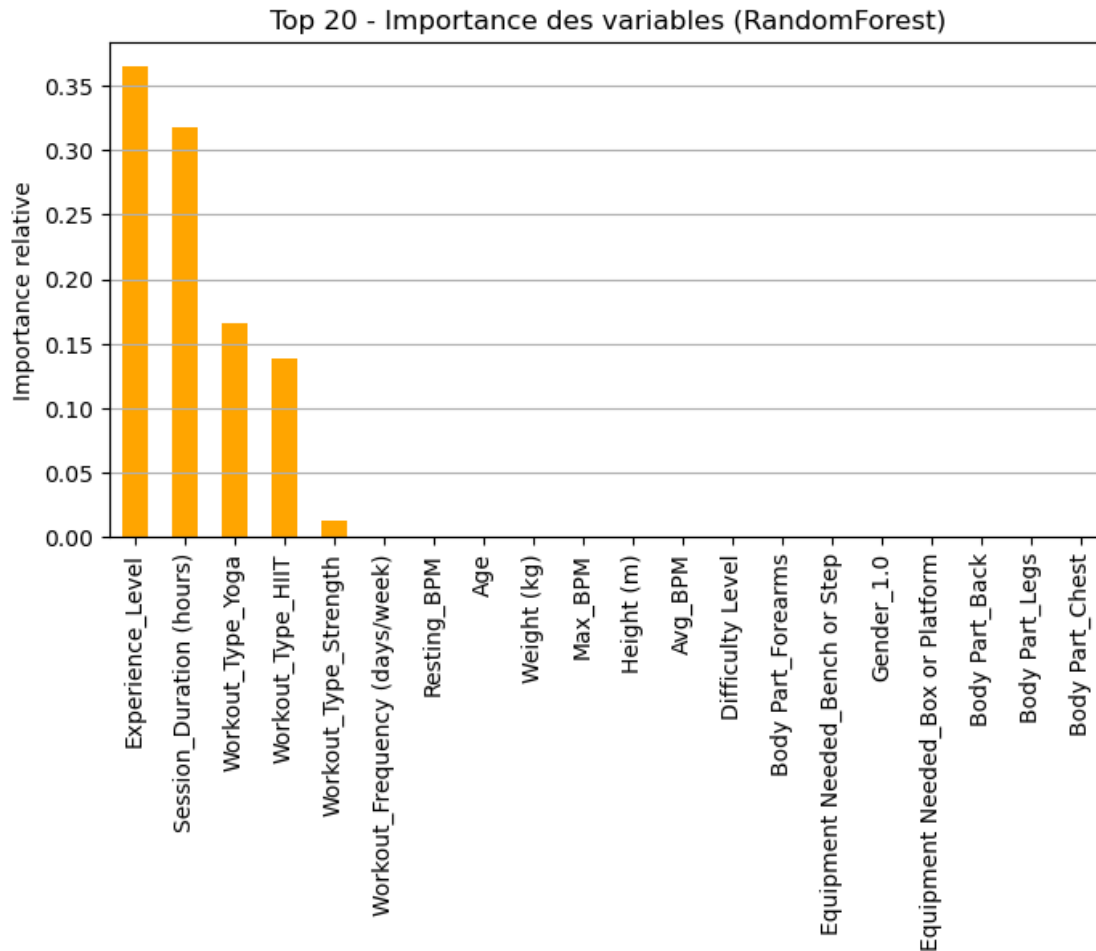
# Modèle RandomForest pour importance des features
model_rf = RandomForestRegressor(random_state=42, n_estimators=100)
model_rf.fit(X_train, y_train)

importances = pd.Series(model_rf.feature_importances_, index=X_train.columns).
    ↪sort_values(ascending=False)
top_importances = importances.head(20)

plt.figure(figsize=(8,4))
top_importances.plot(kind="bar", color="orange")
plt.title("Top 20 - Importance des variables (RandomForest)")
plt.ylabel("Importance relative")
plt.grid(True, axis='y')
plt.show()

```



1.9.3 9.3. Création de nouvelles features

Certaines combinaisons de variables améliorent la capacité prédictive.

On crée ici plusieurs variables dérivées liées à la physiologie et à l'activité physique : - $\text{BPM_effort_ratio} = \text{Avg_BPM} / \text{Max_BPM}$ - $\text{Fat_to_weight_ratio} = \text{Fat_Percentage} / \text{Weight (kg)}$ - $\text{Workout_intensity} = \text{Calories_Burned} / \text{Session_Duration (hours)}$ (proxy de rendement énergétique) - $\text{Hydration_ratio} = \text{Water_Intake (liters)} / \text{Workout_Frequency (days/week)}$

```
[169]: def create_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    # Ratios cardio
    if {"Avg_BPM", "Max_BPM"}.issubset(df.columns):
        df["BPM_effort_ratio"] = df["Avg_BPM"] / (df["Max_BPM"].replace(0, np.
        ↪nan))
        # feature_cols_raw.append("BPM_effort_ratio")
    # Composition corporelle
    if {"Fat_Percentage", "Weight (kg)"}.issubset(df.columns):
```

```

        df["Fat_to_weight_ratio"] = df["Fat_Percentage"] / (df["Weight (kg)"] .
↪replace(0, np.nan))
        # Hydratation relative
        if {"Water_Intake (liters)", "Workout_Frequency (days/week)"} .issubset(df .
↪columns):
            df["Hydration_ratio"] = df["Water_Intake (liters)"] / ↵
↪(df["Workout_Frequency (days/week)"] + 1e-3)
            # Nettoyage inf/NaN générés par divisions
            df = df.replace([np.inf, -np.inf], np.nan)
            df = df.fillna(0)
        return df

```

1.9.4 9.4. Réduction des features redondantes

Certaines variables peuvent être fortement corrélées entre elles, ce qui nuit à la stabilité du modèle.

On applique un filtrage basé sur la matrice de corrélation et un seuil de tolérance pour supprimer les doublons d'information.

```

[170]: # === Gestion du scaling (REUSE, ne pas refitter) ===
train_features = feature_cols_raw.copy()

# Ajouter les features dérivées si elles existent
extra_feats = ["BPM_effort_ratio"]
extra_feats = [c for c in extra_feats if c in X_train.columns]

for c in extra_feats:
    if c not in train_features:
        train_features.append(c)

print("Features utilisées par le modèle :", train_features)

# Extraction des matrices standardisées
X_train_scaled = X_train[train_features]
X_val_scaled = X_val[train_features]
X_test_scaled = X_test[train_features]

# === Analyse de corrélation ===
# Convertir en DataFrame si ce n'est pas déjà le cas
if isinstance(X_train_scaled, np.ndarray):
    X_train_df = pd.DataFrame(X_train_scaled, columns=train_features)
else:
    X_train_df = X_train_scaled.copy()

corr_matrix = X_train_df.corr().abs()
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1) .
↪astype(bool))
to_drop = [c for c in upper_tri.columns if any(upper_tri[c] > 0.95)]

```

```
print(f"Variables supprimées pour corrélation élevée (>0.95) : {len(to_drop)}")

X_train_reduced = X_train_df.drop(columns=to_drop, errors="ignore")
X_val_reduced    = X_val_scaled.drop(columns=to_drop, errors="ignore")
X_test_reduced   = X_test_scaled.drop(columns=to_drop, errors="ignore")
```

Features utilisées par le modèle : ['Age', 'Weight (kg)', 'Height (m)', 'Gender_1.0', 'Experience_Level', 'Difficulty_Level', 'Workout_Frequency (days/week)', 'Session_Duration (hours)', 'Max_BPM', 'Avg_BPM', 'Resting_BPM', 'Workout_Type_HIIT', 'Workout_Type_Strength', 'Workout_Type_Yoga', 'Body Part_Arms', 'Body Part_Back', 'Body Part_Chest', 'Body Part_Forearms', 'Body Part_Legs', 'Body Part_Shoulders', 'Equipment_Needed_Bench or Step', 'Equipment_Needed_Bench or Sturdy Surface', 'Equipment_Needed_Bench, Barbell', 'Equipment_Needed_Box or Platform', 'Equipment_Needed_Cable Machine', 'Equipment_Needed_Cable Machine or Resistance Band', 'Equipment_Needed_Dumbbells', 'Equipment_Needed_Dumbbells or Barbell', 'Equipment_Needed_Kettlebell', 'Equipment_Needed_Low Bar or TRX', 'Equipment_Needed_None or Dumbbell', 'Equipment_Needed_None or Dumbbells', 'Equipment_Needed_Parallel Bars or Chair', 'Equipment_Needed_Pull-up Bar', 'Equipment_Needed_Resistance Band', 'Equipment_Needed_Resistance Band or Cable Machine', 'Equipment_Needed_Step or Box', 'Equipment_Needed_Wall']

Variables supprimées pour corrélation élevée (>0.95) : 0

1.9.5 9.5. Export des nouvelles données

On sauvegarde le dataset enrichi et allégé pour les étapes suivantes de modélisation supervisée.

```
[171]: feature_path = export_dir / "X_train_feature_engineered.parquet"
X_train_reduced.to_parquet(feature_path, index=False)

print(f"Export du jeu de données enrichi : {feature_path}")
print(f"Nombre final de colonnes : {X_train_reduced.shape[1]}")

feature_path = export_dir / "X_test_feature_engineered.parquet"
X_test_reduced.to_parquet(feature_path, index=False)

print(f"Export du jeu de données enrichi : {feature_path}")
print(f"Nombre final de colonnes : {X_test_reduced.shape[1]}")

feature_path = export_dir / "X_val_feature_engineered.parquet"
X_val_reduced.to_parquet(feature_path, index=False)

print(f"Export du jeu de données enrichi : {feature_path}")
print(f"Nombre final de colonnes : {X_val_reduced.shape[1]}")
```

Export du jeu de données enrichi : c:\Users\fbback\Desktop\Projets\Dev\GitHub\tra
in.me\src\data\processed\life_style_data\X_train_feature_engineered.parquet
Nombre final de colonnes : 38

Export du jeu de données enrichi : c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\data\processed\life_style_data\X_test_feature_engineered.parquet
Nombre final de colonnes : 38
Export du jeu de données enrichi : c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\data\processed\life_style_data\X_val_feature_engineered.parquet
Nombre final de colonnes : 38

Critères de validation : - Identification claire des variables les plus influentes sur la cible ; - Création de nouvelles variables pertinentes et explicables ; - Suppression des redondances au-delà d'un seuil de corrélation de 0.95 ; - Aucune valeur manquante introduite lors des transformations.

Cette étape améliore la qualité informationnelle du dataset et prépare le terrain pour la phase 6 – Choix et entraînement du modèle supervisé.

1.10 10. Choix du modèle d'apprentissage supervisé

1.10.1 10.1. Objectif

L'objectif de cette section est de sélectionner et d'entraîner un ou plusieurs modèles supervisés afin de prédire la variable cible `Calories_Burned` à partir des variables explicatives sélectionnées.

Plusieurs algorithmes seront comparés selon leurs performances et leurs caractéristiques : - Régression linéaire : modèle de base simple, interprétable et rapide ; - Random Forest Regressor : modèle non linéaire, robuste et performant sur des données hétérogènes ; - Gradient Boosting Regressor : modèle plus complexe, souvent plus précis mais plus coûteux en calcul.

```
[172]: X_train = pd.read_parquet(export_dir / "X_train_feature_engineered.parquet")
y_train = pd.read_parquet(export_dir / "y_train.parquet").squeeze()
X_test = pd.read_parquet(export_dir / "X_test_feature_engineered.parquet")
y_test = pd.read_parquet(export_dir / "y_test.parquet").squeeze()
X_val = pd.read_parquet(export_dir / "X_val_feature_engineered.parquet")
y_val = pd.read_parquet(export_dir / "y_val.parquet").squeeze()

print(f"Données chargées : {X_train.shape[0]} lignes, {X_train.shape[1]} ↵
      ↵ colonnes")
print(f"Données chargées : {X_test.shape[0]} lignes, {X_test.shape[1]} ↵
      ↵ colonnes")
print(f"Données chargées : {X_val.shape[0]} lignes, {X_val.shape[1]} colonnes")
```

Données chargées : 13992 lignes, 38 colonnes

Données chargées : 2999 lignes, 38 colonnes

Données chargées : 2999 lignes, 38 colonnes

1.10.2 10.2. Modèle de base : Régression linéaire

La régression linéaire sert de point de référence.

Elle permet de mesurer la capacité des features à expliquer la variable cible et sert de baseline pour les modèles plus complexes.

```
[173]: # Baseline linéaire
model_lr = LinearRegression()
model_lr.fit(X_train, y_train_std)
y_pred_lr_std = model_lr.predict(X_val)
print("LR (val) → MAE(std), RMSE(std), R²:",
      mean_absolute_error(y_val_std, y_pred_lr_std),
      np.sqrt(mean_squared_error(y_val_std, y_pred_lr_std)),
      r2_score(y_val_std, y_pred_lr_std))

# Random Forest
model_rf = RandomForestRegressor(random_state=42, n_estimators=300)
model_rf.fit(X_train, y_train_std)
y_pred_rf_std = model_rf.predict(X_val)
print("RF (val) → MAE(std), RMSE(std), R²:",
      mean_absolute_error(y_val_std, y_pred_rf_std),
      np.sqrt(mean_squared_error(y_val_std, y_pred_rf_std)),
      r2_score(y_val_std, y_pred_rf_std))

mae_lr = mean_absolute_error(y_test, y_pred_lr_std)
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr_std))
r2_lr = r2_score(y_test, y_pred_lr_std)

print(f"Régression linéaire → MAE: {mae_lr:.2f}, RMSE: {rmse_lr:.2f}, R²: ␣
      ↪{r2_lr:.3f}")
```

```
LR (val) → MAE(std), RMSE(std), R²: 0.12036921596623128 0.18272653158227822
0.9680578527344643
RF (val) → MAE(std), RMSE(std), R²: 0.001198960310860013 0.002429660996555996
0.9999943525518785
Régression linéaire → MAE: 1.12, RMSE: 1.41, R²: -1.023
```

1.10.3 10.3. Modèle non linéaire : Random Forest

Le Random Forest Regressor capture les relations non linéaires entre les variables.

Il est moins sensible aux outliers et aux distributions non normales.

```
[174]: model_rf = RandomForestRegressor(random_state=42, n_estimators=200, ␣
      ↪max_depth=None)
print(X_train.columns)
model_rf.fit(X_train, y_train)
y_pred_rf = model_rf.predict(X_test)

mae_rf = mean_absolute_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
r2_rf = r2_score(y_test, y_pred_rf)
```

```
print(f"Random Forest → MAE: {mae_rf:.10f}, RMSE: {rmse_rf:.10f}, R²: {r2_rf:.10f}")
```

```
Index(['Age', 'Weight (kg)', 'Height (m)', 'Gender_1.0', 'Experience_Level',
      'Difficulty_Level', 'Workout_Frequency (days/week)',
      'Session_Duration (hours)', 'Max_BPM', 'Avg_BPM', 'Resting_BPM',
      'Workout_Type_HIIT', 'Workout_Type_Strength', 'Workout_Type_Yoga',
      'Body Part_Arms', 'Body Part_Back', 'Body Part_Chest',
      'Body Part_Forearms', 'Body Part_Legs', 'Body Part_Shoulders',
      'Equipment_Needed_Bench or Step',
      'Equipment_Needed_Bench or Sturdy Surface',
      'Equipment_Needed_Bench, Barbell', 'Equipment_Needed_Box or Platform',
      'Equipment_Needed_Cable Machine',
      'Equipment_Needed_Cable Machine or Resistance Band',
      'Equipment_Needed_Dumbbells', 'Equipment_Needed_Dumbbells or Barbell',
      'Equipment_Needed_Kettlebell', 'Equipment_Needed_Low Bar or TRX',
      'Equipment_Needed_None or Dumbbell',
      'Equipment_Needed_None or Dumbbells',
      'Equipment_Needed_Parallel Bars or Chair',
      'Equipment_Needed_Pull-up Bar', 'Equipment_Needed_Resistance Band',
      'Equipment_Needed_Resistance Band or Cable Machine',
      'Equipment_Needed_Step or Box', 'Equipment_Needed_Wall'],
      dtype='object')
```

Random Forest → MAE: 0.0012527293, RMSE: 0.0030232668, R²: 0.9999907632

1.10.4 10.4. Modèle avancé : Bagging Regressor

Le Bagging Regressor repose sur le principe du Bootstrap Aggregating.

Il entraîne en parallèle plusieurs modèles faibles, chacun sur un échantillon tiré avec remise du jeu de données, puis moyenne leurs prédictions.

```
[175]: model_bag = BaggingRegressor(
        n_estimators=200,
        random_state=42,
        n_jobs=-1
    )

model_bag.fit(X_train, y_train)
y_pred_bag = model_bag.predict(X_test)

mae_bag = mean_absolute_error(y_test, y_pred_bag)
rmse_bag = np.sqrt(mean_squared_error(y_test, y_pred_bag))
r2_bag = r2_score(y_test, y_pred_bag)

print(f"Bagging Regressor → MAE: {mae_bag:.10f}, RMSE: {rmse_bag:.10f}, R²: {r2_bag:.10f}")
```

Bagging Regressor → MAE: 0.0012504349, RMSE: 0.0030189013, R²: 0.9999907899

1.10.5 10.5. Modèle avancé : Gradient Boosting

Le Gradient Boosting Regressor ajuste successivement de petits modèles faibles pour corriger les erreurs du précédent.

C'est un excellent compromis entre précision et interprétabilité.

```
[176]: model_gb = GradientBoostingRegressor(random_state=42, n_estimators=300,
    ↪ learning_rate=0.05, max_depth=3)
model_gb.fit(X_train, y_train)
y_pred_gb = model_gb.predict(X_test)

mae_gb = mean_absolute_error(y_test, y_pred_gb)
rmse_gb = np.sqrt(mean_squared_error(y_test, y_pred_gb))
r2_gb = r2_score(y_test, y_pred_gb)

print(f"Gradient Boosting → MAE: {mae_gb:.10f}, RMSE: {rmse_gb:.10f}, R²:
    ↪ {r2_gb:.10f}")
```

Gradient Boosting → MAE: 0.0116458400, RMSE: 0.0154163093, R²: 0.9997598247

1.10.6 10.6. Comparaison des modèles

On compare les modèles selon trois métriques principales : - MAE (Mean Absolute Error) : erreur moyenne absolue ; - RMSE (Root Mean Squared Error) : pénalise davantage les grosses erreurs ; - R² : proportion de variance expliquée par le modèle (plus proche de 1 = meilleur).

```
[177]: results = pd.DataFrame({
    "Modèle": ["Régression Linéaire", "Random Forest", "Bagging Regressor",
    ↪ "Gradient Boosting"],
    "MAE": [mae_lr, mae_rf, mae_bag, mae_gb],
    "RMSE": [rmse_lr, rmse_rf, rmse_bag, rmse_gb],
    "R²": [r2_lr, r2_rf, r2_bag, r2_gb]
})

display(results.sort_values("R²", ascending=False).reset_index(drop=True))
```

	Modèle	MAE	RMSE	R²
0	Bagging Regressor	0.001250	0.003019	0.999991
1	Random Forest	0.001253	0.003023	0.999991
2	Gradient Boosting	0.011646	0.015416	0.999760
3	Régression Linéaire	1.123618	1.414916	-1.023154

1.10.7 10.7. Sélection du meilleur modèle

```
[178]: # --- Sélection automatique du meilleur modèle selon le R² ---

# Mapping noms → objets modèles
models_dict = {
    "Régression Linéaire": model_lr,
```



```

    "Random Forest": model_rf,
    "Bagging Regressor": model_bag,
    "Gradient Boosting": model_gb,
}

# Recherche du nom du meilleur modèle basé sur le  $R^2$ 
best_model_name = results.sort_values("R2", ascending=False).iloc[0]["Modèle"]
best_model = models_dict[best_model_name]

print(f"Meilleur modèle sélectionné : {best_model_name}")
best_model

```

Meilleur modèle sélectionné : Bagging Regressor

[178]: BaggingRegressor(n_estimators=200, n_jobs=-1, random_state=42)

1.10.8 10.8. Gestion du scaling

```

[179]: # X_* sont déjà standardisés par le bloc précédent
X_train_scaled = X_train[feature_cols_raw].values
X_val_scaled    = X_val[feature_cols_raw].values
X_test_scaled   = X_test[feature_cols_raw].values

y_train_scaled = y_train_std # déjà standardisé
y_val_scaled   = y_val_std
y_test_scaled  = y_test_std

```

1.10.9 10.9. Entraînement

```

[180]: # --- Entraînement du modèle sur données normalisées
best_model.fit(X_train_scaled, y_train_scaled)

```

[180]: BaggingRegressor(n_estimators=200, n_jobs=-1, random_state=42)

1.10.10 10.10. Vérifier la variance du dataset

```

[181]: # Sanity check variabilité
import numpy as np
print("Variance moyenne des features :", np.mean(np.var(X_train_scaled,
    ↪axis=0)))
print("Variance de la cible :", np.var(y_train_scaled))
y_check = best_model.predict(X_train_scaled[:10])
print("Prédictions sur 10 échantillons :", np.round(y_check, 4))
y_check_kcal = target_scaler.inverse_transform(y_check.reshape(-1, 1)).ravel()
print("Prédictions sur 10 échantillons (kcal) :", np.round(y_check_kcal, 2))

```

Variance moyenne des features : 1.0

Variance de la cible : 1.0

```
Prédictions sur 10 échantillons : [-0.2253 -0.7921  0.5984  1.1313 -0.102
-0.7241 -0.7775 -0.1179  1.0448
 1.156 ]
Prédictions sur 10 échantillons (kcal) : [1167.02  883.14 1579.65 1846.54
1228.81  917.22  890.45 1220.83 1803.21
1858.9 ]
```

1.10.11 10.11. Vérification des features

```
[182]: feature_names = X_train[best_model.feature_names_in_].columns.tolist() \
        if hasattr(best_model, "feature_names_in_") else X_train.columns.tolist()

        setattr(best_model, "feature_names_in_", np.array(feature_names, dtype=object))
        print(list(best_model.feature_names_in_))
```

```
['Age', 'Weight (kg)', 'Height (m)', 'Gender_1.0', 'Experience_Level',
'Difficulty Level', 'Workout_Frequency (days/week)', 'Session_Duration (hours)',
'Max_BPM', 'Avg_BPM', 'Resting_BPM', 'Workout_Type_HIIT',
'Workout_Type_Strength', 'Workout_Type_Yoga', 'Body Part_Arms', 'Body
Part_Back', 'Body Part_Chest', 'Body Part_Forearms', 'Body Part_Legs', 'Body
Part_Shoulders', 'Equipment Needed_Bench or Step', 'Equipment Needed_Bench or
Sturdy Surface', 'Equipment Needed_Bench, Barbell', 'Equipment Needed_Box or
Platform', 'Equipment Needed_Cable Machine', 'Equipment Needed_Cable Machine or
Resistance Band', 'Equipment Needed_Dumbbells', 'Equipment Needed_Dumbbells or
Barbell', 'Equipment Needed_Kettlebell', 'Equipment Needed_Low Bar or TRX',
'Equipment Needed_None or Dumbbell', 'Equipment Needed_None or Dumbbells',
'Equipment Needed_Parallel Bars or Chair', 'Equipment Needed_Pull-up Bar',
'Equipment Needed_Resistance Band', 'Equipment Needed_Resistance Band or Cable
Machine', 'Equipment Needed_Step or Box', 'Equipment Needed_Wall']
```

1.10.12 10.12. Sauvegarde des scalers pour être en phase avec “test_model.ipynb” et Gradio

```
[183]: # (optionnel) petite sanity-check sur 10 prédictions test
        # On utilise le même espace de features que pour l'entraînement
        y_test_pred_std = best_model.predict(X_test_scaled)
        y_test_pred_kcal = target_scaler.inverse_transform(y_test_pred_std.reshape(-1, 1))
        print("Extrait prédictions (kcal):", np.round(y_test_pred_kcal[:10], 2))
```

```
c:\Users\fbback\Desktop\Projets\Dev\GitHub\train.me\.conda\Lib\site-
packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid
feature names, but BaggingRegressor was fitted with feature names
  warnings.warn(

Extrait prédictions (kcal): [1689.63 1047.54 1208.87 2201.31  712.34  536.14
1583.16  880.58 1809.92
1038.01]
```

1.10.13 10.13. Sauvegarde du modèle sélectionné

```
[184]: # Injecter les noms de features (ordre)
import numpy as np
setattr(best_model, "feature_names_in_", np.array(feature_names, dtype=object))

# Sauvegarde
joblib.dump(best_model, model_dir / "model.joblib")
print("Modèle et scalers sauvegardés dans :", model_dir)
```

Modèle et scalers sauvegardés dans :

c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\models\v1\life_style_data

1.10.14 10.14. Validation : chargement et fusion X_val + y_val (en unités réelles)

```
[185]: X_val = pd.read_parquet(export_dir / "X_val_feature_engineered.parquet")
y_val = pd.read_parquet(export_dir / "y_val.parquet").squeeze()

df_val = pd.concat([X_val, y_val], axis=1)
```

1.10.15 10.15. Export vers SQLite pour inspection manuelle

```
[186]: con = sqlite3.connect(export_dir / "life_style_data_val.db")
df_val.to_sql("validation_set", con, if_exists="replace", index=False)
con.close()

print(f"Fusion et export terminés : {df_val.shape[0]} lignes, {df_val.shape[1]} ↵
↵ colonnes")
```

Fusion et export terminés : 2999 lignes, 39 colonnes

1.10.16 10.16. Informations relatives au choix du modèle (son nom et les métriques associées)

```
[187]: # Détection "friendly" du type de modèle
def model_friendly_name(est):
    if isinstance(est, RandomForestRegressor):
        return "Random Forest"
    if isinstance(est, BaggingRegressor):
        return "Bagging Regressor"
    if isinstance(est, GradientBoostingRegressor):
        return "Gradient Boosting"
    if isinstance(est, LinearRegression):
        return "Régression Linéaire"
    return est.__class__.__name__

# Métriques par modèle (cast en float pour le JSON)
metrics = {
```

```

    "Régression Linéaire": {
        "MAE": float(mae_lr), "RMSE": float(rmse_lr), "R2": float(r2_lr)
    },
    "Random Forest": {
        "MAE": float(mae_rf), "RMSE": float(rmse_rf), "R2": float(r2_rf)
    },
    "Bagging Regressor": {
        "MAE": float(mae_bag), "RMSE": float(rmse_bag), "R2": float(r2_bag)
    },
    "Gradient Boosting": {
        "MAE": float(mae_gb), "RMSE": float(rmse_gb), "R2": float(r2_gb)
    }
}

# Infos sur le modèle sélectionné
model_filename = "model.joblib" # correspond à ton dump: joblib.
↪ dump(best_model, model_dir / "model.joblib")
model_path = (model_dir / model_filename).resolve()

best_info = {
    "model_type": model_friendly_name(best_model),
    "model_class": best_model.__class__.__name__,
    "model_path": str(model_path),
    "params": best_model.get_params()
}

# Contexte d'évaluation
report = {
    "created_at": datetime.utcnow().isoformat() + "Z",
    "target": y.name if hasattr(y, "name") else "Calories_Burned",
    "n_features": int(X_test.shape[1]) if "X_test" in globals() else None,
    "n_test_samples": int(len(y_test)) if "y_test" in globals() else None,
    "metrics_by_model": metrics,
    "selected_model": best_info
}

# Sauvegarde
report_path = model_dir / "model_report.json"
with open(report_path, "w", encoding="utf-8") as f:
    json.dump(report, f, ensure_ascii=False, indent=2)

print(f"Rapport JSON enregistré : {report_path}")

```

Rapport JSON enregistré : c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\models\v1\life_style_data\model_report.json

1.10.17 10.17. Plan de features

```
[188]: # Spécifications "métier" optionnelles (bornes & enums).
# Celles-ci PRIMENT sur l'inférence automatique depuis X_train.
manual_specs = {
    "Age": {"type": "integer", "min": 10, "max": 90},
    "Gender": {"type": "string", "enum": ["Male", "Female", "Other", "Prefer_
↳not to say"]},
    "Weight (kg)": {"type": "number", "min": 30.0, "max": 200.0},
    "Height (m)": {"type": "number", "min": 1.50, "max": 2.30},
    # Exemple si tu ajoutes d'autres champs plus tard :
    "Max_BPM": {"type": "number", "min": 159, "max": 200},
    "Avg_BPM": {"type": "number", "min": 119, "max": 170},
    "Resting_BPM": {"type": "number", "min": 49.0, "max": 75.0},
    "Session_Duration (hours)": {"type": "number", "min": 0.1, "max": 6},
    "Workout_Type": {"type": "string", "enum":_
↳["Cardio", "Strength", "HIIT", "Yoga", "Pilates", "Other"]},
    "Experience_Level": {"type": "number", "min": 1.0, "max": 3.05},
    "Workout_Frequency (days/week)": {"type": "number", "min": 1.94, "max": 5.
↳06},
    "Difficulty Level": {
        "type": "integer",
        "enum": [0, 1, 2],
        "min": 0,
        "max": 2
    },
    "Body Part": {
        "type": "string",
        "enum": ["Abs", "Arms", "Back", "Chest", "Forearms", "Legs",_
↳"Shoulders"],
    },
    "Equipment Needed": {                                # nouveau champ
        "type": "string",
        "enum": [
            "Step or Box",
            "Parallel Bars or Chair",
            "Bench or Sturdy Surface",
            "None or Dumbbells",
            "Resistance Band",
            "Wall",
            "None or Dumbbell",
            "Dumbbells",
            "Dumbbells or Barbell",
            "Low Bar or TRX",
            "Cable Machine",
            "Box or Platform",
            "Bench or Chair",
```

```

        "Resistance Band or Cable Machine",
        "Kettlebell",
        "Bench or Step",
        "Pull-up Bar",
        "Barbell",
        "Cable Machine or Resistance Band"
    ]
},
}

# On s'assure que la cible n'est JAMAIS dans selected_features
target_col = "Calories_Burned"
selected_features = [f for f in selected_features if f != target_col]

# Vérification de cohérence
if target_col in selected_features:
    raise ValueError(f"La cible {target_col} ne doit pas figurer dans les_
↳ features sélectionnées.")
else:
    print(f"Colonne cible '{target_col}' exclue de la liste des features_
↳ ({len(selected_features)} restantes).")

# Fonction pour inférer le type et les contraintes à partir de X_train_
↳ (fallback)
def infer_field_spec(name: str, series: pd.Series) -> dict:
    # dtype → type JSON
    if pd.api.types.is_integer_dtype(series):
        ftype = "integer"
    elif pd.api.types.is_float_dtype(series):
        ftype = "number"
    else:
        ftype = "string"

    spec = {"type": ftype}

    if ftype in {"integer", "number"}:
        # bornes observées (n'anéantit pas manual_specs)
        s = series.dropna()
        if not s.empty:
            spec["min_observed"] = float(np.nanmin(s))
            spec["max_observed"] = float(np.nanmax(s))
    else:
        # enum observée (capée à 50 valeurs)
        uniques = series.dropna().astype(str).unique()
        if 0 < len(uniques) <= 50:
            spec["enum_observed"] = sorted(map(str, uniques))

```

```

    return spec

# Construire le schéma (priorité aux specs manuelles)
# On part du X_train de l'expérience courante
train_cols = [c for c in selected_features if c in X_train.columns]
schema_features = []

for col in selected_features:
    if col in manual_specs:
        spec = {"name": col, **manual_specs[col], "required": True}
    elif col in train_cols:
        spec = {"name": col, **infer_field_spec(col, X_train[col]), "required": True}
    else:
        # colonne absente du train → type par défaut string
        spec = {"name": col, "type": "string", "required": True}

    schema_features.append(spec)

# Exemple de payload (médiane/mode depuis X_train, sinon placeholders)
example = {}
for f in schema_features:
    name, ftype = f["name"], f["type"]
    if name in X_train.columns:
        s = X_train[name]
        if ftype in {"integer", "number"}:
            val = float(s.median()) if s.notna().any() else (f.get("min") or 0)
            if ftype == "integer":
                val = int(round(val))
            example[name] = val
        else:
            # string
            if "enum" in f:
                example[name] = f["enum"][0]
            elif s.dropna().size > 0:
                example[name] = str(s.dropna().iloc[0])
            else:
                example[name] = ""
    else:
        # fallback si la colonne n'existe pas en train
        if ftype in {"integer", "number"}:
            example[name] = f.get("min", 0)
        else:
            example[name] = f.get("enum", [""])[0] if f.get("enum") else ""

# Objet final du schéma
feature_schema = {

```

```

    "model_name": "life_style_data",
    "model_version": "v2.12-minimal", # ajuste si tu es sur un set élargi
    "created_at": datetime.utcnow().isoformat() + "Z",
    "target": y.name if isinstance(y, pd.Series) else "Calories_Burned",
    "features": schema_features, # ordre = contrat d'entrée
    "ordering_guarantee": True, # l'ordre des features est garanti
    "one_hot_handle_unknown": "ignore", # convention côté pipeline
    "notes": "Schéma d'entrée pour l'inférence REST. Les features dérivées/
↳ encodages sont calculées côté backend.",
    "example_payload": example
}

# Sauvegarde
schema_path = model_dir / "feature_schema.json"
with open(schema_path, "w", encoding="utf-8") as f:
    json.dump(feature_schema, f, ensure_ascii=False, indent=2)

print(f"Schéma d'entrée sauvegardé : {schema_path.name}")

```

Colonne cible 'Calories_Burned' exclue de la liste des features (14 restantes).
Schéma d'entrée sauvegardé : feature_schema.json

Critères de validation : - Modèle entraîné sans erreur ni surapprentissage excessif ; - Comparaison objective des métriques (MAE, RMSE, R^2) ; - Sauvegarde du meilleur modèle pour l'étape d'optimisation.

Cette section valide le choix du modèle de base et ouvre la voie à la phase 7 — Entraînement, validation et évaluation des performances.

1.10.18 10.18. Sélection du meilleur modèle

Le modèle offrant le meilleur R^2 et la plus faible erreur (MAE/RMSE) est sélectionné pour les étapes suivantes : - interprétation des résultats, - optimisation des hyperparamètres (section suivante), - et déploiement dans le pipeline complet.

1.11 11. Code complet avec pipeline

```

[189]: # pipeline = Pipeline(steps=[
#     ('imputer', SimpleImputer(strategy='median')),
#     ('scaler', StandardScaler()),
#     ('model', LinearRegression())
# ])

# # Pipeline générique
# pipeline = Pipeline(steps=[
#     ("features", FunctionTransformer(create_features, validate=False)),
#     ("scaler", "passthrough"), # sera remplacé par StandardScaler pour les
↳ modèles qui en ont besoin
#     ("model", RandomForestRegressor(random_state=42)) # placeholder

```



```
# ])
```

```
[190]: # # Grille de modèles & hyperparamètres
# param_grid = [
#     # A) Random Forest (pas de scaling requis)
#     {
#         "scaler": ["passthrough"],
#         "model": [RandomForestRegressor(random_state=42)],
#         "model__n_estimators": [200, 400],
#         "model__max_depth": [None, 10, 20],
#         "model__min_samples_split": [2, 5]
#     },
#     # B) Gradient Boosting (pas de scaling requis)
#     {
#         "scaler": ["passthrough"],
#         "model": [GradientBoostingRegressor(random_state=42)],
#         "model__n_estimators": [200, 300],
#         "model__learning_rate": [0.05, 0.1],
#         "model__max_depth": [2, 3]
#     },
#     # C) Régression linéaire (scaling nécessaire)
#     {
#         "scaler": [StandardScaler(with_mean=True, with_std=True)],
#         "model": [LinearRegression()]
#         # Pas d'hyperparamètres pour LinearRegression standard
#     }
# ]

# # CV + métriques
# cv = KFold(n_splits=5, shuffle=True, random_state=42)

# grid = GridSearchCV(
#     estimator=pipeline,
#     param_grid=param_grid,
#     scoring={"rmse": "neg_root_mean_squared_error", "r2": "r2"},
#     refit="r2", # on conserve le meilleur modèle au sens du R²
#     cv=cv,
#     n_jobs=-1,
#     verbose=1
# )
```

```
[191]: # # pipeline.fit(X_train, y_train)
# grid.fit(X_train, y_train)
# print("Best params:", grid.best_params_)
# print("Best CV R² :", grid.best_score_)
```

```
[192]: ## y_pred = pipeline.predict(X_test)
## pd.DataFrame(y_pred, columns=['Predicted_PRICE'])

# best_model = grid.best_estimator_
## y_pred = best_model.predict(X_test)

## mae = mean_absolute_error(y_test, y_pred)
## rmse = np.sqrt(mean_squared_error(y_test, y_pred))
## r2 = r2_score(y_test, y_pred)

## print(f"Test → MAE: {mae:.2f} | RMSE: {rmse:.2f} | R²: {r2:.3f}")

## 2.2 Appliquer EXACTEMENT les mêmes features aux deux
# df_x = pd.DataFrame(X_val)
# print(df_x)

# y_pred = best_model.predict(X_val)
## test_pred = best_model.predict(X_test)
## print(f"Test R²: {r2_score(y_test, test_pred):.3f}")

# mae = mean_absolute_error(y_test, y_pred)
# rmse = np.sqrt(mean_squared_error(y_test, y_pred))
# r2 = r2_score(y_test, y_pred)

# print(f"Test → MAE: {mae:.2f} | RMSE: {rmse:.2f} | R²: {r2:.3f}")

# df_val = pd.DataFrame(y_pred)
# df_val
## df_val = pd.DataFrame(y_pred, columns=['Predicted_PRICE'])
## y_pred
```

```
[193]: ## Modèle
# model_path = model_dir / "best_pipeline_model.joblib"
# joblib.dump(best_model, model_path)

## Petit rapport
# report = {
#     "best_params": grid.best_params_,
#     "cv_best_r2": float(grid.best_score_),
#     "test_mae": float(mae),
#     "test_rmse": float(rmse),
#     "test_r2": float(r2),
#     "n_features_in": int(X_train.shape[1])
# }
# pd.Series(report).to_json(export_dir / "best_pipeline_model_report.json",
↪    indent=2)
```

```
# print(f" Modèle sauvegardé → {model_path}")
# print(" Rapport → best_supervised_model_report.json")
```

Conseils d’usage / bonnes pratiques (rapides) - Zéro fuite de données : le scaler est dans la pipeline, appris uniquement sur le train. - Même feature engineering partout : FunctionTransformer garantit l’application identique sur train/test. - Comparaison saine : trois familles de modèles testées, avec une grille raisonnable pour démarrer. - Prochain step : section “7. Entraînement, validation, évaluation des performances” → on étendra la grille (ou on basculera sur RandomizedSearchCV / Optuna) et on ajoutera des courbes d’apprentissage et permutation importances.

1.12 12. Conversion df_val to SQLite

```
[194]: # # Chargement
# X_feature_val = pd.read_parquet(export_dir / "X_val_feature_engineered.
↳ parquet")
# X_val = pd.read_parquet(export_dir / "X_val.parquet")
# y_val = pd.read_parquet(export_dir / "y_val.parquet").squeeze()

# # Connexion SQLite
# # con = sqlite3.connect(export_dir / "life_style_data_val.db")

# # Fusion sur position (même ordre)
# df_val = pd.concat([X_feature_val, y_val], axis=1)

# # Export
# df_val.to_sql("validation_feature_set", con, if_exists="replace", index=False)

# # Fusion sur position (même ordre)
# df_val = pd.concat([X_val, y_val], axis=1)

# # Export
# df_val.to_sql("validation_set", con, if_exists="replace", index=False)

# con.close()

# print(f" Fusion et export terminés : {df_val.shape[0]} lignes, {df_val.
↳ shape[1]} colonnes")
```